spectral_analysis_walk_through

May 18, 2026

1 Spectral analysis: How?

1.0.1 1. We first need to collect our data, whether it is observations or model data.

[1]:
```
import xarray as xr
import pandas as pd
%pylab inline
```

```
%pylab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib
```

[2]:
```
model = xr.open_mfdataset('/ADB/data/DUACS/1*/*/*.nc') # an ocean model
gauge = pd.read_csv("/ADB/data/tide-gauges/GESLA_3/
  ↪wellington-071a-nzl-uhslc",skiprows=41,header=None,
                    delim_whitespace=True) # tide gauge from New Zealand
```

```
/tmp/ipykernel_23442/519212513.py:2: FutureWarning: The 'delim_whitespace'
keyword in pd.read_csv is deprecated and will be removed in a future version.
Use ``sep='\s+'`` instead
  gauge = pd.read_csv("/ADB/data/tide-gauges/GESLA_3/wellington-071a-nzl-
uhslc",skiprows=41,header=None,
```

2 inspect the data

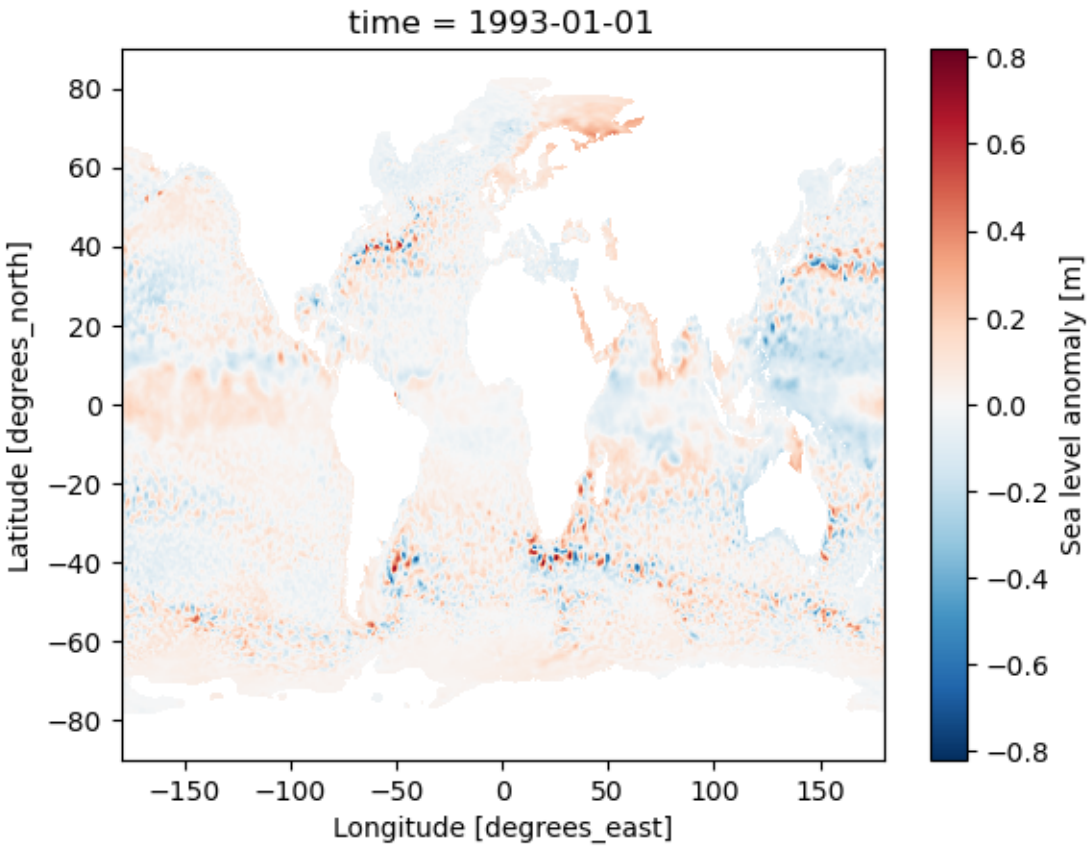
[3]:
```
model
```

[3]:
```
<xarray.Dataset> Size: 212GB
Dimensions:          (time: 2556, latitude: 720, longitude: 1440, nv: 2)
Coordinates:
  * latitude         (latitude) float32 3kB -89.88 -89.62 -89.38 … 89.62 89.88
  * longitude        (longitude) float32 6kB -179.9 -179.6 -179.4 … 179.6 179.9
  * nv               (nv) int32 8B 0 1
  * time             (time) datetime64[ns] 20kB 1993-01-01 … 1999-12-31
Data variables: (12/14)
    adt              (time, latitude, longitude) float64 21GB
```

```
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    crs             (time) int32 10kB -2147483647 -2147483647 … -2147483647
    err_sla         (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    err_ugosa       (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    err_vgosa       (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    flag_ice        (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    …                …
    sla             (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    tpa_correction  (time) float64 20kB dask.array<chunksize=(1,),
meta=np.ndarray>
    ugos            (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    ugosa           (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    vgos            (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
    vgosa           (time, latitude, longitude) float64 21GB
dask.array<chunksize=(1, 50, 50), meta=np.ndarray>
Attributes: (12/45)
    Conventions:                    CF-1.6
    Metadata_Conventions:           Unidata Dataset Discovery v1.0
    cdm_data_type:                  Grid
    comment:                        Sea Surface Height measured by Altimetry…
    contact:                        servicedesk.cmems@mercator-ocean.eu
    creator_email:                  servicedesk.cmems@mercator-ocean.eu
    …                              …
    time_coverage_duration:         P1D
    time_coverage_end:              1993-01-01T12:00:00Z
    time_coverage_resolution:       P1D
    time_coverage_start:            1992-12-31T12:00:00Z
    title:                          DT merged all satellites Global Ocean Gr…
    NCO:                            4.7.2
```

[4]:
```
model.sla[0].plot()
```

[4]:
```
<matplotlib.collections.QuadMesh at 0x7f770e93ecf0>
```



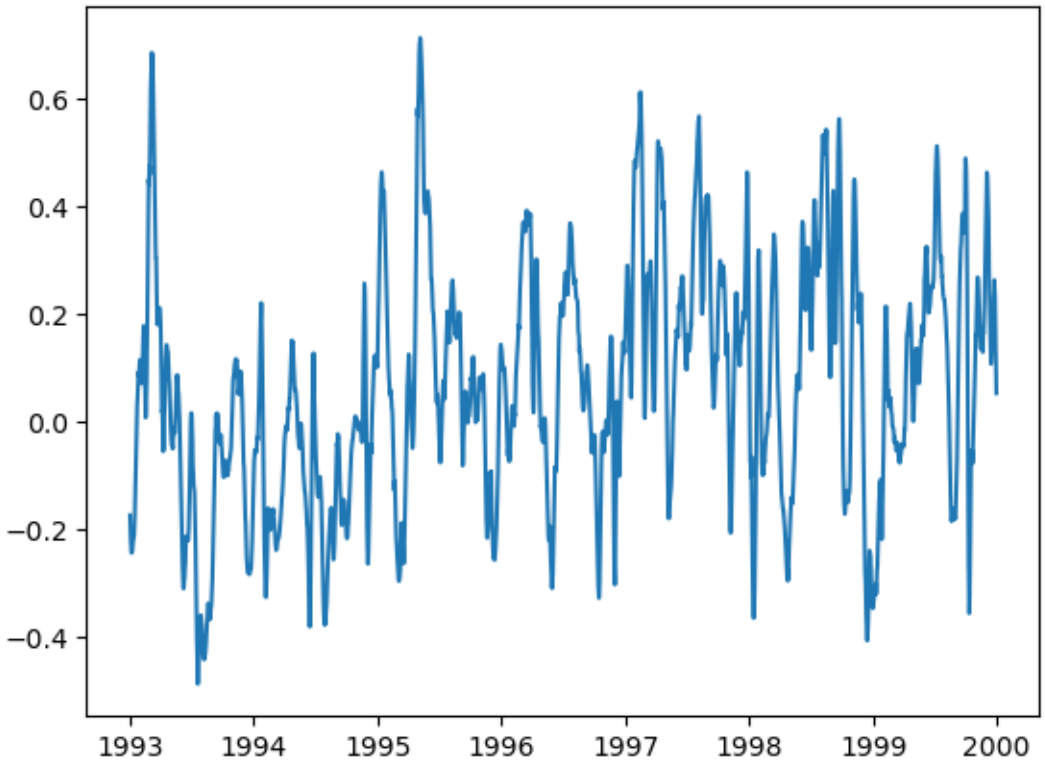
2.0.1 2. lets extract one point on the model

[5]:
```
model_subset = model.sel(latitude=-36.875,longitude=18.625)
```

[6]:
```
model_data = model_subset.sla.values
```

[7]:
```
plt.plot(model_subset.time,model_data)
```

[7]:
```
[<matplotlib.lines.Line2D at 0x7f76ae7c1950>]
```



3 the tide gauge

[8]:
```
gauge # 3 and 4 are quality flags, we SHOULD take these into account
```

[8]:

	0	1	2	3	4
0	1944/11/18	00:00:00	0.885	1	1
1	1944/11/18	01:00:00	0.845	1	1
2	1944/11/18	02:00:00	0.925	1	1
3	1944/11/18	03:00:00	1.035	1	1
4	1944/11/18	04:00:00	1.265	1	1
…	…	…	…		
649711	2018/12/31	07:00:00	0.586	1	1
649712	2018/12/31	08:00:00	0.790	1	1
649713	2018/12/31	09:00:00	0.979	1	1
649714	2018/12/31	10:00:00	1.220	1	1
649715	2018/12/31	11:00:00	1.562	1	1

[649716 rows x 5 columns]

[9]:
```
gauge_quali = gauge.iloc[np.where((gauge[3]==1) & (gauge[4]
↪==1)|(gauge[2]>-90))].reset_index(drop=True)
```

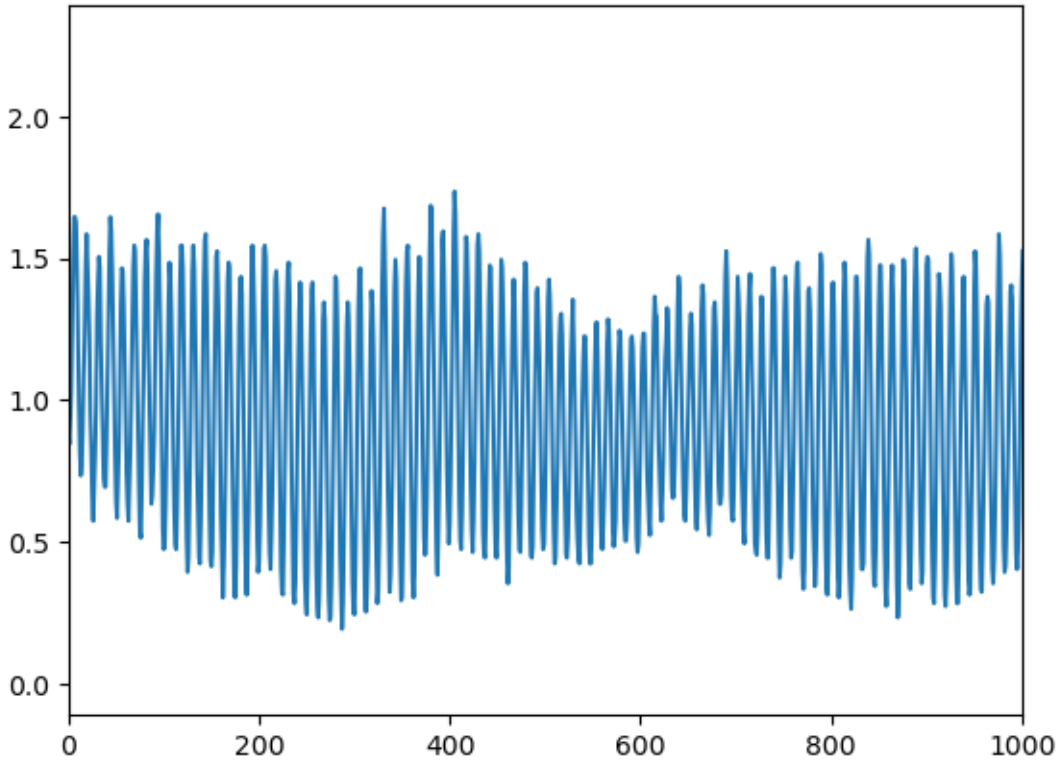
[10]:
```
print('not using quality flag, data points =', len(gauge))
print('not using quality flag, data points =', len(gauge_quali))
```

```
not using quality flag, data points = 649716
not using quality flag, data points = 632175
```

[11]:
```
plt.plot(gauge_quali[2])

plt.xlim(0,1000)
```

[11]:
```
(0.0, 1000.0)
```



3.0.1 3. Implementing the FFT

Tide Gauge

[ ]:
```

```

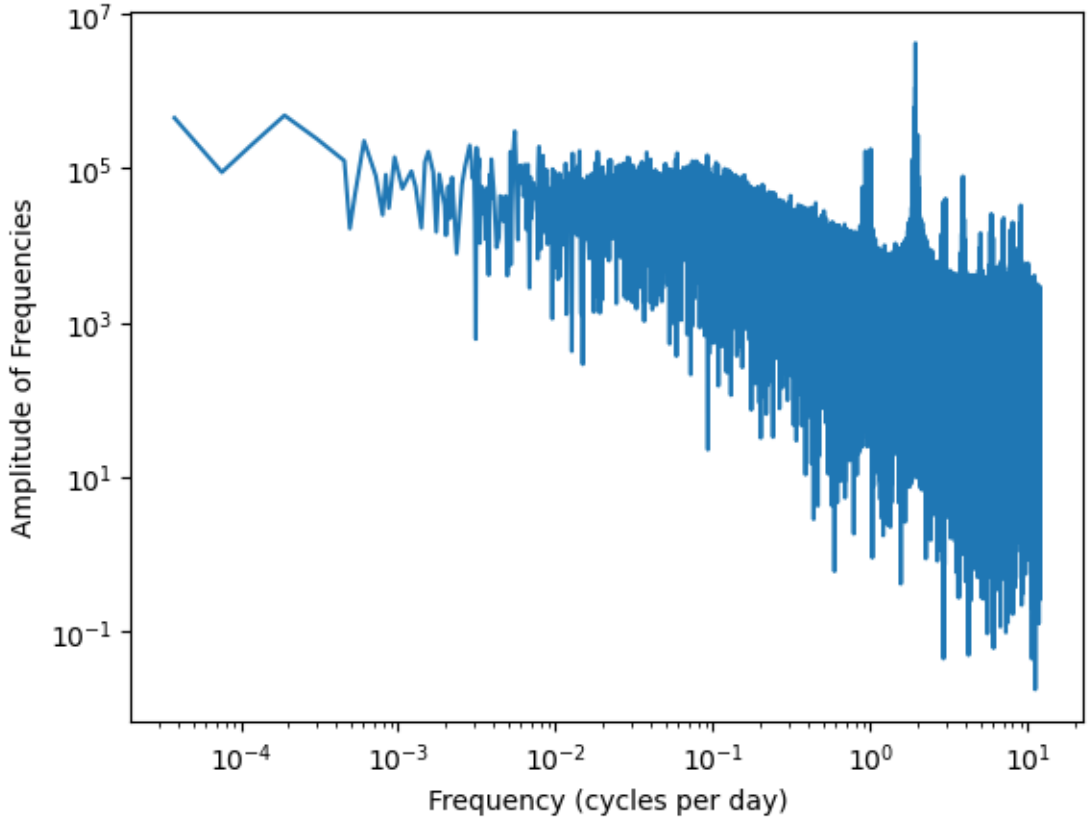
[12]:
```
from scipy.fft import fft, fftfreq

data = gauge_quali[2] *100 #  - np.nanmean(gauge_quali[2])  # I want to only␣
 ↪look at anomalies, to match model
N = len(data)                                          # total length of gauge␣
 ↪data series
fs = 1                                                 # my sample frequency is␣
 ↪hourly
dt = 1.0 / fs                                          # time step [hourly in the␣
 ↪case of our gauges]


X = fft(data) # using the FFT from Scipy, also available in numpy
freqs = fftfreq(N, d=dt)
mask = np.where((X>0)&(freqs>0))
plt.plot(freqs[mask]*24,X[mask])
plt.ylabel("Amplitude of Frequencies")
plt.xlabel("Frequency (cycles per day)")
plt.semilogy() # i do this to reveal more detail
plt.semilogx() # same as above
```

```
/home/hartdavis/.local/share/mamba/envs/PY38/lib/python3.13/site-
packages/matplotlib/cbook.py:1762: ComplexWarning: Casting complex values to
real discards the imaginary part
  return math.isfinite(val)
/home/hartdavis/.local/share/mamba/envs/PY38/lib/python3.13/site-
packages/matplotlib/cbook.py:1398: ComplexWarning: Casting complex values to
real discards the imaginary part
  return np.asarray(x, float)
```

[12]:
```
[]
```



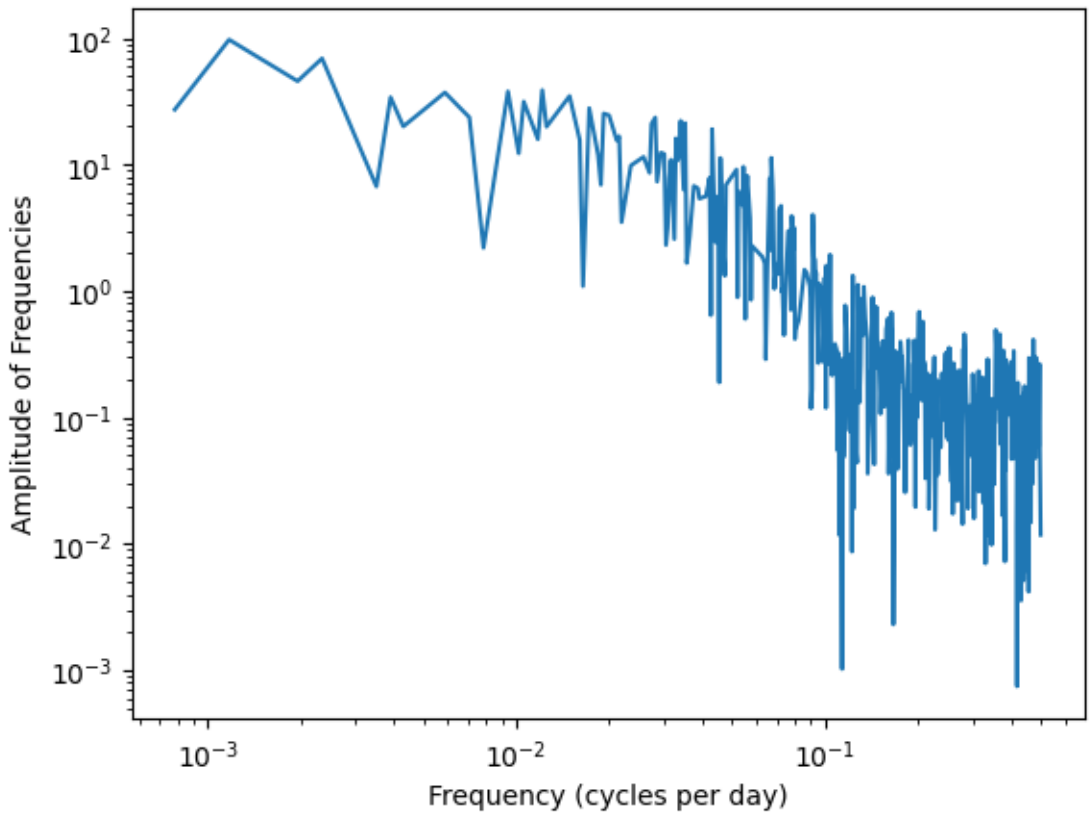
Model

[13]:
```
from scipy.fft import fft, fftfreq

data = model_data
N = len(data)         # total length of gauge data series
fs = 1                # my sample frequency is daily
dt = 1.0 / fs         # time step

# Apply FFT
X = fft(data)
freqs = fftfreq(N, d=dt)
mask = np.where((X>0)&(freqs>0))
plt.plot(freqs[mask],X[mask])
plt.ylabel("Amplitude of Frequencies")
plt.xlabel("Frequency (cycles per day)")
plt.semilogy() # i do this to reveal more detail
plt.semilogx() # same as above
```

[13]: []



[ ]:

3.0.2 Power Spectral Density (PSD)

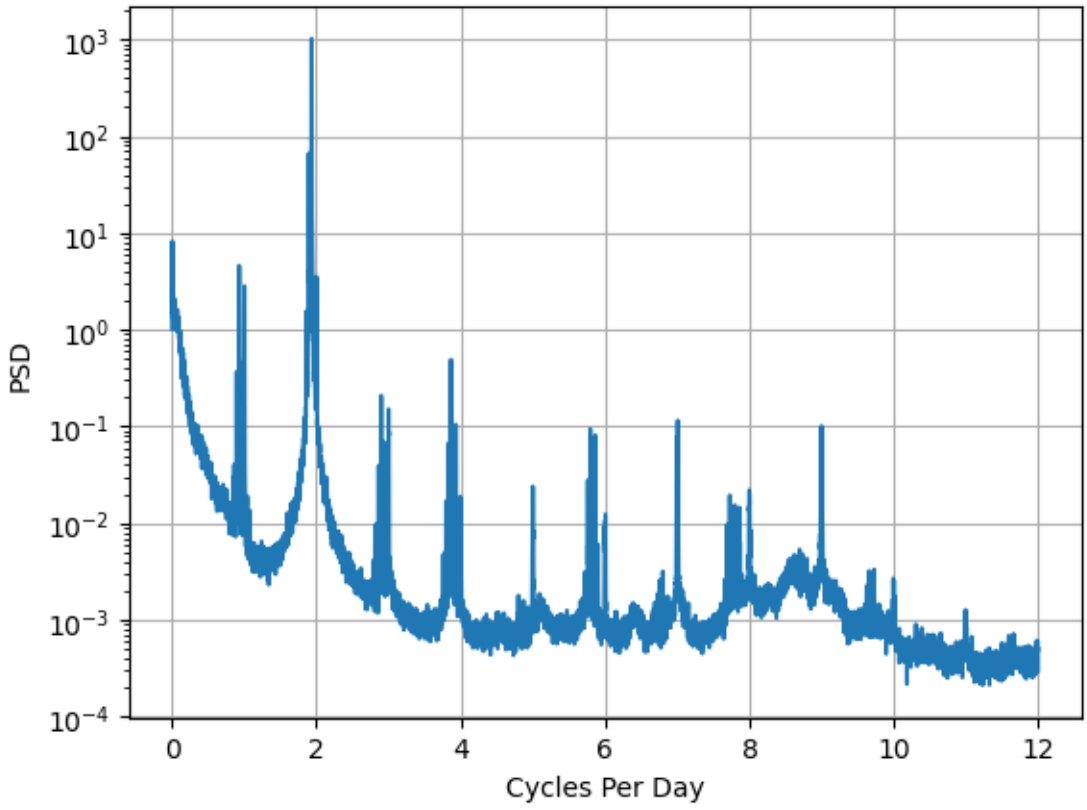
[24]:
```
from scipy import signal

data = gauge_quali[2] - np.nanmean(gauge_quali[2])  # I want to only look at␣
 ↪anomalies, to match model

freqs, psd = signal.welch(data,fs = 1,nperseg=25000)

plt.plot(freqs*24,psd)
plt.xlabel('Cycles Per Day')
plt.ylabel('PSD')

# plt.semilogx()
plt.semilogy()
plt.grid()
```


PSD
10^{3}
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-4}
0
2
4
6
8
10
12
Cycles Per Day

[ ]:

[ ]:

[ ]: